Linear regression

Course of Machine Learning

Master Degree in Computer Science
University of Rome "Tor Vergata"

a.a. 2025-2026

Linear Models and Basis Function Expansion

The foundation of many statistical learning algorithms lies in the use of linear models. At their core, these models assume that the target prediction can be expressed as a linear combination of the input features. This assumption, while seemingly restrictive, provides the basis for powerful and interpretable mathematical frameworks.

In its simplest form, a linear model predicts an output by scaling each input feature x_i by a weight w_i and adding a constant offset. Mathematically, for a d-dimensional input vector $\mathbf{x} = (x_1, \dots, x_d)^T$, the hypothesis function h is defined as:

$$h(\mathbf{x}; w_1, \dots, w_d, b) = w_1 x_1 + w_2 x_2 + \dots + w_d x_d + b$$

Here, the constant term b is known as the bias. Its role is crucial: it allows the model to make a non-zero prediction even when all input features are zero, effectively shifting the decision surface away from the origin.

To simplify the notation, we typically represent the weights as a vector $\mathbf{w} \in \mathbb{R}^d$. The model can then be written compactly using a dot product:

$$h(\mathbf{x}; \mathbf{w}, b) = \mathbf{w}^T \mathbf{x} + b = \begin{pmatrix} w_1 & w_2 & \cdots & w_d \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{pmatrix} + b$$

Commentary on Linearity: It is essential to observe that these models are linear in two distinct ways. First, they are linear with respect to the **features** $\mathbf{x}$, describing a flat hyperplane in the input space. Second, and more importantly for optimization purposes, they are linear with respect to the **parameters** $\mathbf{w}$ and b. Since learning a model involves treating parameters as variables to be minimized or maximized, this linearity ensures that the resulting objective functions (such as sum-of-squares error) often have a convex shape, making the global optimum easy to find.

Feature Engineering and Basis Functions

While the basic linear model is restricted to linear relationships in the original input space, we can greatly expand its expressive power through feature engineering. This is achieved by mapping the original input $\mathbf{x} \in \mathbb{R}^d$ into a new, typically higher-dimensional space $\mathbb{R}^m$ using a set of predefined basis functions $\{\phi_1, \dots, \phi_m\}$.

The transformation is defined as $\boldsymbol{\phi}(\mathbf{x}) = (\phi_1(\mathbf{x}), \dots, \phi_m(\mathbf{x}))^T$. By applying this mapping, we transform a prediction task that might be non-linear in $\mathbb{R}^d$ into a task that is linear in $\mathbb{R}^m$. The modified model structure becomes:

$$h(\mathbf{x}; \mathbf{w}, b) = \sum_{j=1}^{m} w_j \phi_j(\mathbf{x}) + b$$

It is a vital point that applying basis functions *does not change* the linearity of the model with respect to its parameters $\mathbf{w}$. We are still performing a linear combination; we have simply changed the "ingredients" of that combination.

Common Types of Basis Functions

The choice of basis functions determines the type of patterns the model can recognize. We generally categorize them into global and local functions:

- **Polynomial Basis Functions (Global):** These are defined as $\phi_j(x) = x^j$. They are "global" because a change in a single weight w_j affects the prediction across the entire range of the input x. While powerful, they can be unstable at the boundaries of the data.

$$\phi_j(x) = x^j$$

- **Gaussian Basis Functions (Local):** These take the form of a bell curve. They are "local" because the function $\phi_j(x)$ is only significantly non-zero in a small region around its mean μ_j.

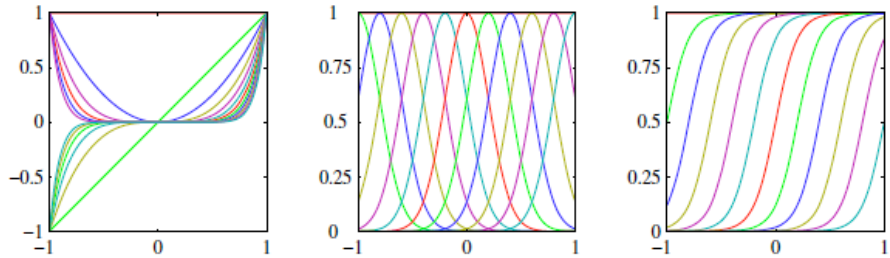
$$\phi_j(x) = \exp\left(-\frac{(x-\mu_j)^2}{2s^2}\right)$$

- **Sigmoid Basis Functions (Local):** The sigmoid function $\sigma(\cdot)$ maps the input to a range between 0 and 1. It represents a "step-like" transition centered at μ_j.

$$\phi_j(x) = \sigma\left(\frac{x-\mu_j}{s}\right) = \frac{1}{1+e^{-\frac{x-\mu_j}{s}}}$$

- **Hyperbolic Tangent Basis Functions (Local):** Similar to the sigmoid but mapping to the range $[-1, 1]$, the $\tanh$ function is often used in neural networks and signal processing.

$$\phi_j(x) = \tanh(x) = 2\sigma(x) - 1 = \frac{1-e^{-\frac{x-\mu_j}{s}}}{1+e^{-\frac{x-\mu_j}{s}}}$$



In a practical training scenario, we deal with a dataset of n observations, represented by the matrix $\mathbf{X}$:

$$\mathbf{X} = \begin{pmatrix} - & \mathbf{x}_1 & - \\ & \vdots & \\ - & \mathbf{x}_n & - \end{pmatrix} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nd} \end{pmatrix}$$

When we apply our basis functions $\boldsymbol{\phi}$ to every sample in the dataset, we construct the **Design Matrix** $\boldsymbol{\Phi}$. Each row of this matrix corresponds to the transformed representation of a single data point, and each column represents the evaluation of a specific basis function across all samples:

$$\boldsymbol{\Phi} = \begin{pmatrix} \phi_1(\mathbf{x}_1) & \phi_2(\mathbf{x}_1) & \cdots & \phi_m(\mathbf{x}_1) \\ \phi_1(\mathbf{x}_2) & \phi_2(\mathbf{x}_2) & \cdots & \phi_m(\mathbf{x}_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_1(\mathbf{x}_n) & \phi_2(\mathbf{x}_n) & \cdots & \phi_m(\mathbf{x}_n) \end{pmatrix}$$

Moving forward, for the sake of simplicity, we will often refer to the original training set $(\mathbf{X}, \mathbf{t})$. However, it is intended that all derivations and considerations apply equally to the transformed dataset $\boldsymbol{\Phi}$. Whether we are working in the original feature space or a mapped basis space, the underlying mechanics of linear optimization remain identical.

Suppose we are provided with a set of n observations involving two real-valued variables, x (the input or independent variable) and t (the target or dependent variable). This collection is represented as a set of pairs:

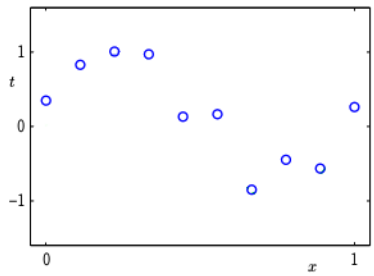
$$\{(x_1, t_1), (x_2, t_2), \dots, (x_n, t_n)\}$$

Our primary objective is **generalization**: we wish to exploit these specific observations to predict, for any new or unseen value $\tilde{x}$, the most likely corresponding value of the target variable t.

In mathematical terms, we represent the training set as a pair of vectors:

- $\mathbf{x} = (x_1, \dots, x_n)^T$, representing the inputs.
- $\mathbf{t} = (t_1, \dots, t_n)^T$, representing the observed targets.

These vectors are assumed to be related through an unknown deterministic rule, or function, which has been obscured by stochastic elements.

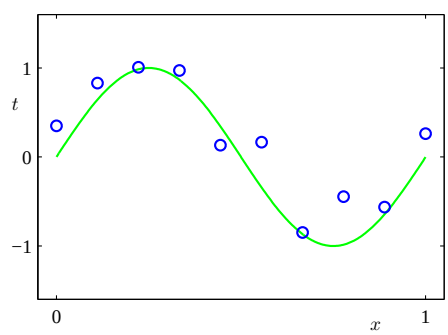


To understand how to model this data, we must first make an assumption about how the data was generated. In this specific scenario, we assume that the "true" but hidden relationship between x and t is defined by a sine wave: $t = \sin(2\pi x)$.

However, real-world data is rarely pristine. We assume the observations in our training set are corrupted by **Gaussian noise**. This noise, denoted by ε, represents measurement errors or unobserved latent variables. We model this as a random variable with a mean of 0 and a specific variance σ^2, such that $\varepsilon \sim \mathcal{N}(\varepsilon; 0, \sigma^2)$. Consequently, each observed target t_i is expressed as:

$$t_i = \sin(2\pi x_i) + \varepsilon_i$$

Our goal is not to fit the noise, but to "see through" it to approximate the original deterministic function $t = \sin(2\pi x)$ as closely as possible, relying solely on the noisy evidence provided in the training set.



One of the most versatile ways to approximate an unknown function is through **polynomial regression**. In this approach, we choose to model the relationship using a polynomial of a fixed degree $m > 0$. The hypothesis function $h(x; \mathbf{w}, b)$ takes the form:

$$h(x; \mathbf{w}, b) = w_1 x + w_2 x^2 + \ldots + w_m x^m + b = \sum_{j=1}^{m} w_j x^j + b$$

where $\mathbf{w} = (w_1, \ldots, w_m)^T$ represents the vector of coefficients and b is the bias or intercept (effectively w_0).

This model can be interpreted as a specific instance of a **linear model** operating in a transformed feature space. By defining a set $\boldsymbol{\phi}$ of m basis functions such that $\phi_j(x) = x^j$ for $j = 1, \ldots, m$, we can rewrite the hypothesis as:

$$h(x; \mathbf{w}, b) = \sum_{j=1}^{m} w_j \phi_j(x) + b$$

The choice of the degree m is critical. A low value of m (e.g., $m = 1$, a straight line) might be too simple to capture the oscillations of the sine wave, leading to underfitting. Conversely, an excessively high value of m might allow the polynomial to oscillate wildly to pass through every noisy data point, leading to overfitting.

The central challenge of the learning phase is to compute the optimal values for $\mathbf{w}$ and b that minimize the discrepancy between our predictions $h(x_i)$ and the observed targets t_i.

Mathematical Optimization: Convexity and Global Minima

To find the optimal parameters $\overline{\mathbf{w}}^*$, we must find the point where the gradient of $E(\overline{\mathbf{w}})$ is zero. A critical advantage of the quadratic loss is that it is a **convex function**. Since the total error $E(\overline{\mathbf{w}})$ is the sum of n convex functions, $E(\overline{\mathbf{w}})$ is itself convex.

In a convex landscape, any local minimum is guaranteed to be a **global minimum**. Furthermore, because the error function is quadratic, its derivative is linear. Solving for the point where this linear derivative equals zero yields a unique solution $\overline{\mathbf{w}}^*$, which defines our final predictor $h(\mathbf{x}; \overline{\mathbf{w}}^*)$.

To derive the optimal parameters analytically, we calculate the partial derivatives of the error function with respect to each weight and the bias:

$$\frac{\partial E(\overline{\mathbf{w}})}{\partial w_k} = 2\sum_{i=1}^{n} r_i(\overline{\mathbf{w}})\frac{\partial}{\partial w_k} r_i(\overline{\mathbf{w}}) = 2\sum_{i=1}^{n}\left(\sum_{j=1}^{d} w_j x_{ij} + b - t_i\right) x_{ik} \quad k = 1, \dots, d$$

$$\frac{\partial E(\overline{\mathbf{w}})}{\partial b} = 2\sum_{i=1}^{n} r_i(\overline{\mathbf{w}})\frac{\partial}{\partial b} r_i(\overline{\mathbf{w}}) = 2\sum_{i=1}^{n}\left(\sum_{j=1}^{d} w_j x_{ij} + b - t_i\right)$$

This follows because $\frac{\partial}{\partial w_k} r_i(\overline{\mathbf{w}}) = x_{ik}$ (the value of the k-th feature for the i-th item) and $\frac{\partial}{\partial b} r_i(\overline{\mathbf{w}}) = 1$. Setting these $d + 1$ equations to zero creates a linear system of equations that can be written in matrix form:

$$\overline{\mathbf{X}}\overline{\mathbf{w}} = \mathbf{t}$$

where $\overline{\mathbf{X}}$ is the **augmented design matrix** including a leading column of ones to account for the bias:

$$\overline{\mathbf{X}} = \begin{pmatrix} 1 & x_{11} & \cdots & x_{1d} \\ 1 & x_{21} & \cdots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & \cdots & x_{nd} \end{pmatrix}$$

Assuming the matrix $\overline{\mathbf{X}}^T\overline{\mathbf{X}}$ is non-singular (which is generally true unless the data features are perfectly collinear), the unique solution is given by the **normal equations**:

$$\overline{\mathbf{w}}^* = (\overline{\mathbf{X}}^T\overline{\mathbf{X}})^{-1}\overline{\mathbf{X}}^T\mathbf{t}$$

Numerical Optimization: Gradient Descent

While the closed-form solution is mathematically elegant, it can be computationally expensive for very large datasets due to the matrix inversion. In such cases, we employ **gradient descent**, an iterative numerical method that "walks" down the error surface toward the minimum.

1. **Initialization:** We start with an initial guess $\overline{\mathbf{w}}^{(0)}$, which results in an initial error $E(\overline{\mathbf{w}}^{(0)})$.
2. **Iterative Update:** At each step s, we modify the current parameters $\overline{\mathbf{w}}^{(s-1)}$ in the direction of the **steepest descent**. This direction is the negative of the gradient $\nabla E(\overline{\mathbf{w}})$.
3. **Update Rule:** The parameters are updated using a **learning rate** η:

$$\overline{\mathbf{w}}^{(s)} := \overline{\mathbf{w}}^{(s-1)} - \eta \nabla E(\overline{\mathbf{w}})\Big|_{\overline{\mathbf{w}}^{(s-1)}}$$

Expanding this for the individual weights and the bias, we get:

$$w_k^{(s)} := w_k^{(s-1)} - 2\eta \sum_{i=1}^{n} r_i(\overline{\mathbf{w}}^{(s-1)}) x_{ik}, \quad b^{(s)} := b^{(s-1)} - 2\eta \sum_{i=1}^{n} r_i(\overline{\mathbf{w}}^{(s-1)})$$

In matrix notation, the update looks like this:

$$\overline{\mathbf{w}}^{(s)} := \overline{\mathbf{w}}^{(s-1)} - 2\eta \sum_{i=1}^{n} r_i(\overline{\mathbf{w}}^{(s-1)}) \overline{\mathbf{x}}_i$$

where $\overline{\mathbf{x}}_i$ is the i-th row of the augmented design matrix. This update rule is highly intuitive: the change applied to the parameters is proportional to the residual r_i. If the model predicts an item perfectly, the residue is zero and no update occurs. If the error is large, the update is correspondingly large, pushing the model to correct its prediction for that specific data point.

Consider the task of fitting a polynomial to a set of data points. This process involves applying a set of basis functions $\boldsymbol{\phi} = (1, x, x^2, \dots, x^M)$ to one-dimensional items. Here, x^j represents the input x raised to the j-th power, transforming our one-dimensional input into an $(M+1)$-dimensional space.

The choice of the polynomial degree M is a fundamental example of **model selection**. By assigning a specific value to M, we determine the internal representation of the items in our training set. This decision directly dictates the specific model we use because M defines the number of coefficients in the weight vector $\mathbf{w}$ that must be estimated.

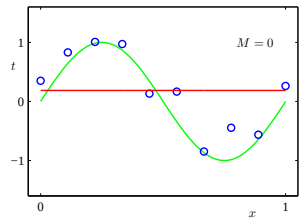
As we increase the complexity of the model by raising M, we provide the model with more degrees of freedom. This allows the polynomial curve to bend more flexibly, resulting in a better approximation of the training set items and a subsequent decrease in training error. However, this flexibility is a double-edged sword:

- When M is small (e.g., $M = 0$ or $M = 1$), the model might be too rigid to capture the underlying patterns, such as the oscillations of a sine wave.
- If the complexity increases to the point where $M + 1 = n$ (where n is the number of training samples), the model has enough parameters to pass exactly through every data point. This achieves a null training error, but it leads to the phenomenon of **overfitting**.

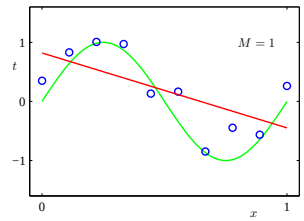
The core issue with overfitting is the failure of *generalization*. While the hypothesis function $h(\mathbf{x}; \mathbf{w}, b)$ is derived exclusively from items in the training set, its true value lies in its ability to provide accurate predictions for new, unseen items across the whole domain.

If $h(\mathbf{x}; \mathbf{w}, b)$ is derived as a too much accurate depiction of the training set—essentially memorizing the noise rather than learning the signal—it results in an unsuitable generalization. In such cases, the model becomes "overly faithful" to the specific samples in the training set, losing the ability to predict the trend for items it has not encountered before.

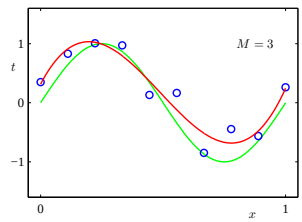
To observe this effect quantitatively in our example, we follow a specific experimental setup:



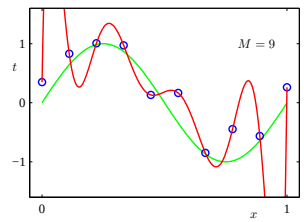
(a) $M = 0$: Underfitting



(b) $M = 1$: Linear fit



(c) $M = 3$: Balanced fit



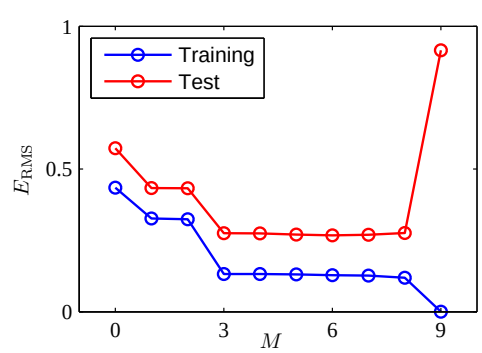
(d) $M = 9$: Overfitting

1. Data Generation: We create a training set $\mathcal{T}_{train}$ of 1-dimensional items. These are generated by uniformly sampling x in the interval $[0, 1]$ and sampling noise ε from a Gaussian distribution $\mathcal{N}(0, \sigma^2)$. The target is computed as $t = \sin(2\pi x) + \varepsilon$.
2. Comparative Setup: We generate a separate test set $\mathcal{T}_{test}$ using the exact same probabilistic process. This set serves as our "unseen data."
3. Measurement: For each degree M under consideration:
   - We derive the optimal weights $\mathbf{w}^*$ by minimizing the empirical risk on the training set $\overline{\mathcal{R}}_{\mathcal{T}_{train}}(\mathbf{w})$.
   - We then evaluate how these weights perform on the test set by computing the Root Mean Square (RMS) error:

$$E_{RMS}(\mathbf{w}^*, \mathcal{T}_{test}) = \sqrt{\overline{\mathcal{R}}_{\mathcal{T}_{test}}(\mathbf{w}^*)} = \sqrt{\frac{1}{|\mathcal{T}_{test}|} \sum_{(x_i,t_i)\in\mathcal{T}_{test}} \left(\sum_{j=0}^{M} w_j^* x_i^j - t_i \right)^2}$$

Note that since our basis set includes $\phi_0(x) = 1$, the coefficient w_0 effectively acts as the bias b; thus, b is implicitly handled within the weight vector $\mathbf{w}^*$. A lower value of $E_{RMS}(\mathbf{w}^*, \mathcal{T}_{test})$ indicates that the model has successfully generalized beyond the training data.

By plotting the E_{RMS} with respect to the degree M for both the training and test sets, we can identify the optimal complexity for our example.



The resulting curves provide two crucial insights:

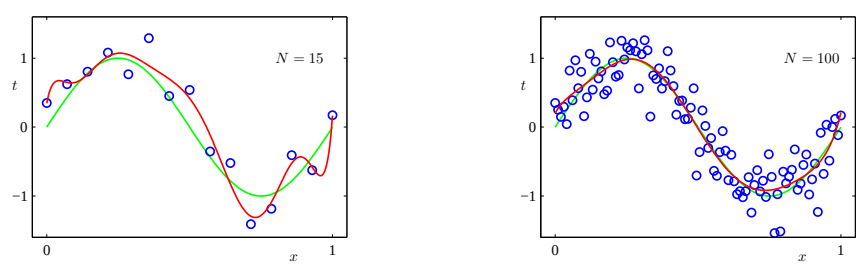
- As M increases, the error on the training set steadily decreases, eventually tending toward 0. This confirms that a more complex model can always "fit" a fixed set of points more closely.
- On the test set, the behavior is non-monotonic. Initially, the error decreases because the added complexity allows the model to capture the sinusoidal nature of the data. However, as M continues to increase, the error begins to rise sharply. This occurs because the model becomes too dependent on the specific noise and distribution of the training set, leading to poor performance on any other data.

Regularization in Linear Models

The phenomenon of overfitting is not solely a function of model complexity, but rather an interplay between the model's degrees of freedom and the quantity of information provided by the data. As we shall see, increasing the amount of evidence is one way to mitigate overfitting, while regularization provides a mathematical framework to control it directly when data is limited.

The Impact of Training Set Size

A critical observation in machine learning is that for a fixed model complexity—for instance, a polynomial of a specific degree M—the tendency to overfit decreases as the dimension (size) of the training set increases. When more data points are available, the model is forced to satisfy a larger number of constraints, making it much harder for a complex curve to "wiggle" through every individual point while still capturing the true underlying signal.



(a) Small dataset: high overfitting. **(b)** Large dataset: reduced overfitting.

This relationship suggests a fundamental rule of thumb: the larger the training set, the higher the acceptable complexity of the model. In a sense, "Big Data" allows us to utilize more sophisticated models without the immediate fear of capturing only noise. However, in many practical scenarios, obtaining more data is expensive or impossible, necessitating techniques to limit model complexity explicitly.

Regularization: Controlling Complexity via the Cost Function

In order to limit the complexity of the model without necessarily reducing the number of parameters, a common and highly effective approach is the introduction of a **regularization term** into the cost function. The total objective to be minimized becomes:

$$E_{total}(\overline{\mathbf{w}}) = E_D(\overline{\mathbf{w}}) + \lambda E_W(\overline{\mathbf{w}})$$

In this expression, we have a clear division of labor between two terms:

- $E_D(\overline{\mathbf{w}})$ represents the empirical risk (the data term). This part is dependent on both the dataset and the parameters, measuring how well the model fits the training observations.
- $E_W(\overline{\mathbf{w}})$ is the **regularization term**, which depends solely on the parameters $\overline{\mathbf{w}}$. Its purpose is to penalize "extreme" parameter values, effectively discouraging the model from becoming overly complex.

The **regularization coefficient** λ serves as a "knob" that controls the relative importance of these two terms. A small λ prioritizes fitting the data (risking overfitting), while a large λ prioritizes keeping the weights small (risking underfitting).

Ridge Regression: Quadratic Regularization

When $E_D(\overline{\mathbf{w}})$ is defined using the quadratic loss, we enter the realm of regularized least squares. The most common form of regularization involves the sum of the squared values of the coefficients, often scaled by $1/2$ for mathematical convenience:

$$E_W(\overline{\mathbf{w}}) = \frac{1}{2}\overline{\mathbf{w}}^T\overline{\mathbf{w}} = \frac{1}{2}\sum_{i=1}^{m} w_i^2 + \frac{b^2}{2}$$

The resulting overall loss function to be minimized is:

$$E(\mathbf{w}) = \frac{1}{2}\sum_{i=1}^{n} r_i(\mathbf{w})^2 + \frac{\lambda}{2}\mathbf{w}^T\mathbf{w} = \frac{1}{2}\boldsymbol{r}(\mathbf{w})^T\boldsymbol{r}(\mathbf{w}) + \frac{\lambda}{2}\mathbf{w}^T\mathbf{w}$$

where $\boldsymbol{r}(\overline{\mathbf{w}})$ is the vector of residues. We can express this vector compactly using the augmented design matrix $\overline{\mathbf{X}}$, the parameter vector $\overline{\mathbf{w}}$, and the target vector $\mathbf{t}$:

$$\boldsymbol{r}(\overline{\mathbf{w}}) = \overline{\mathbf{X}}\overline{\mathbf{w}} - \mathbf{t} = \begin{pmatrix} 1 & x_{11} & \cdots & x_{1d} \\ 1 & x_{21} & \cdots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & \cdots & x_{nd} \end{pmatrix}\begin{pmatrix} b \\ w_1 \\ \vdots \\ w_d \end{pmatrix} - \begin{pmatrix} t_1 \\ t_2 \\ \vdots \\ t_n \end{pmatrix}$$

This specific approach is famously known as **ridge regression** (or Tikhonov regularization). Its primary advantage is that it still allows for a closed-form solution. By setting the gradient of $E(\mathbf{w})$ to zero, we obtain the regularized normal equations:

$$\overline{\mathbf{w}} = (\lambda\mathbf{I} + \overline{\mathbf{X}}^T\overline{\mathbf{X}})^{-1}\overline{\mathbf{X}}^T\mathbf{t}$$

Notice how $\lambda\mathbf{I}$ effectively "strengthens" the diagonal of the matrix before inversion, ensuring the matrix is well-conditioned even if the features are highly correlated.

Generalized Regularization and Lasso

We can generalize the regularization term by treating the power of the summed coefficients as a parameter q:

$$E(\overline{\mathbf{w}}) = \frac{1}{2}\sum_{i=1}^{n} r_i(\overline{\mathbf{w}})^2 + \frac{\lambda}{2}\sum_{j=1}^{d} |w_j|^q + \frac{|b|^q}{2}$$

The case where $q = 1$ is particularly significant and is known as the **Lasso** (Least Absolute Shrinkage and Selection Operator). Lasso regression has the fascinating property of favoring **sparse models**. Unlike Ridge regression, which shrinks weights toward zero but rarely makes them exactly zero, Lasso frequently returns parameter vectors with many null values. This makes Lasso not just a regularization technique, but also a built-in method for feature selection, as it effectively "ignores" less relevant features by zeroing out their corresponding weights.

The behavior of regularized models can be understood by viewing the minimization of the cost function as a **constrained optimization** problem. Minimizing $E(\overline{\mathbf{w}}) = E_D(\overline{\mathbf{w}}) + \lambda E_W(\overline{\mathbf{w}})$ is equivalent to minimizing the empirical risk $E_D(\overline{\mathbf{w}})$ subject to a budget constraint on the total "size" of the weights, defined by $E_W(\overline{\mathbf{w}}) \leq \eta$, where η is inversely related to λ.

The shape of the constraint region depends entirely on the power q used in the regularization term:

- In Ridge regularization ($q = 2$) the constraint $\sum w_j^2 \leq \eta$ defines a **hypersphere** (a circle in 2D). The boundary is smooth and isotropic, meaning it penalizes all directions in the parameter space equally.
- In Lasso regularization ($q = 1$) the constraint $\sum |w_j| \leq \eta$ defines a **hyperdiamond** (a diamond/rhombus in 2D). This shape is characterized by sharp "corners" or vertices that lie exactly on the coordinate axes.

The optimal solution $\overline{\mathbf{w}}^*$ is found at the point where the elliptical level curves of the empirical risk $E_D(\overline{\mathbf{w}})$ (which represent different levels of training error) first touch the boundary of the constraint region.

- Because the Ridge circle is smooth, the likelihood of the elliptical error contours touching the circle exactly on an axis is nearly zero. Consequently, Ridge regression tends to shrink all coefficients towards zero, but they typically remain non-zero. It retains all features, even if their influence is greatly diminished.
- Because the Lasso diamond has "points" on the axes, the elliptical error contours are highly likely to hit one of these vertices first. When the contact occurs at a vertex, the coordinate corresponding to the other axis becomes exactly zero.

This "vertex-hitting" property is why Lasso is celebrated for producing sparse models. By setting irrelevant feature weights to exactly zero, Lasso performs automatic feature selection, resulting in models that are simpler to interpret and more computationally efficient.

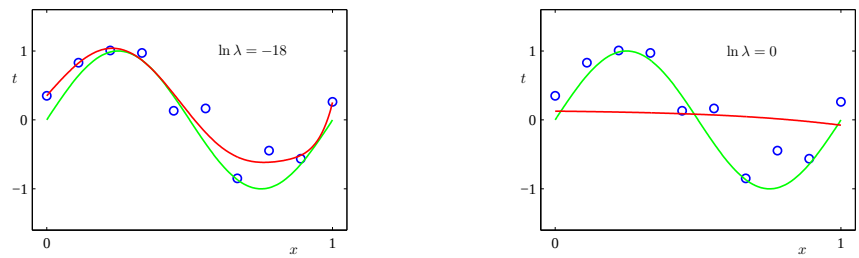
By choosing between $q = 1$ and $q = 2$, we are not just changing a mathematical constant; we are choosing whether we want a model that balances all inputs or one that actively identifies and discards the noise.

Regularization functions by augmenting the standard error function with a penalty term that discourages the coefficients from reaching extreme values. In this example, we apply Ridge regularization with the objective is to minimize a cost function that balances the fit to the data with the magnitude of the weights:

$$E(\overline{\mathbf{w}}) = \frac{1}{2}\sum_{i=1}^{n} r_i(\overline{\mathbf{w}})^2 + \frac{\lambda}{2}\sum_{k=1}^{M} w_k^2 + \frac{b^2}{2} = \frac{1}{2}\sum_{i=1}^{n} r_i(\overline{\mathbf{w}})^2 + \frac{\lambda}{2}\|\overline{\mathbf{w}}\|^2$$

Here, $r_i(\overline{\mathbf{w}})$ is the residual for the i-th observation, and $\|\overline{\mathbf{w}}\|^2$ is the squared Euclidean norm of the parameter vector. The inclusion of this penalty term serves a specific purpose: it effectively "shrinks" the possible values of the coefficients. By limiting the absolute magnitude of w_k, we prevent the polynomial from developing the sharp, high-frequency oscillations required to fit random noise, thereby forcing a smoother, more stable solution.

The behavior of our model is governed by the **regularization coefficient** λ, a hyperparameter that dictates the trade-off between the data-fitting term and the penalty term.



(a) High λ: over-smoothed fit. **(b)** Low λ: high-frequency wiggles.

The relationship between λ and the model's error is typically characterized by three distinct regimes:

- Small λ: The penalty is negligible. The model prioritizes the training data, resulting in a very small training error but a large error on the test set due to significant overfitting.
- Large λ: The penalty dominates. The coefficients are suppressed so heavily that the influence of the actual data values decreases. This leads to a model that is too simple (e.g., nearly a flat line), resulting in a high error on both the training and test sets.
- Intermediate λ: A balance is struck. The error on the training set is moderate, but the error on the test set is minimized, representing the best generalization to unseen data.

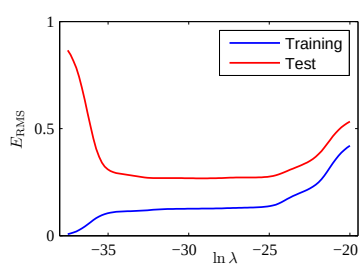


Figura 4. Error on training (blue) and test (red) sets as a function of $\ln \lambda$.

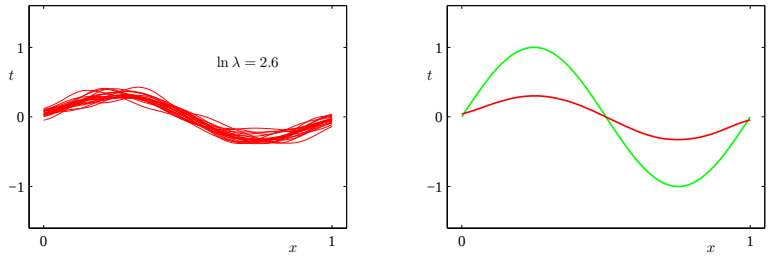
To deepen our interpretation of how regularization affects the model, let us consider a more complex scenario involving the target function $f(x) = \sin(2\pi x)$. We assume that we have $L = 100$ independent training sets $\mathcal{T}^{(1)}, \dots, \mathcal{T}^{(L)}$, each containing $n = 25$ data points.

For each training set, we fit a model using $m = 24$ Gaussian basis functions $\boldsymbol{\phi} = (\phi_1(x), \dots, \phi_m(x))$. This high number of basis functions makes the model extremely prone to overfitting if left unregularized. For each dataset $\mathcal{T}^{(i)}$, we derive a specific prediction function $h^{(i)}(x)$ by minimizing the regularized cost function:

$$E(\mathbf{w}) = \frac{1}{2}\sum_{i=1}^{n} r_i(\overline{\mathbf{w}})^2 + \frac{\lambda}{2}\sum_{k=0}^{m} w_k^2$$

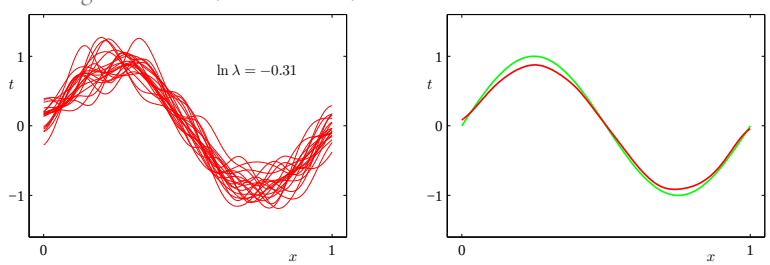
We can now observe how the choice of λ affects the ensemble of these 100 models.

1. Case 1: High Regularization ($\ln \lambda = 2.6$).



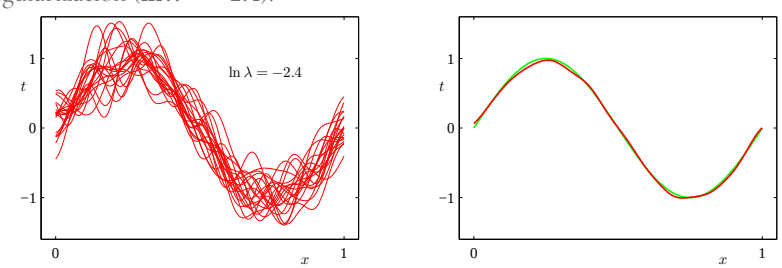
   In the left plot, we see the individual prediction functions $h^{(i)}(x)$. They are very similar to one another, indicating a **low variance**: the specific choice of training set does not change the model much. However, the right plot shows that their average (expectation) is a very poor approximation of the original sine wave. This is a state of **high bias**. The model is too rigid to learn the true signal.

2. Case 2: Intermediate Regularization ($\ln \lambda = -0.31$).



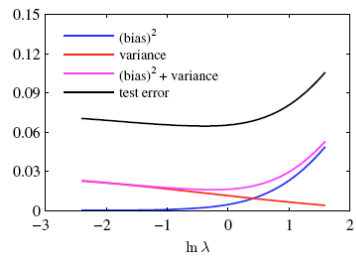
   As we decrease λ, the models gain more flexibility. They begin to follow the sinusoidal trend more closely, and the average of the models provides a much better fit to the underlying function.

3. Case 3: Low Regularization ($\ln \lambda = -2.4$).



   When λ is very small, we observe the opposite of Case 1. The individual models $h^{(i)}(x)$ are wildly different from one another (high variance), as each one overfits the specific noise in its own dataset $\mathcal{T}^{(i)}$. While their average might be quite close to the sine wave (low bias), any single model is likely to provide a poor prediction for new data points.

By plotting the squared bias, the variance, and their sum as functions of λ, we can clearly see the fundamental trade-off of supervised learning.



- The bias reflects the error introduced by approximating a real-life problem with a simplified model. As λ increases (making the model simpler), bias increases.
- The variance reflects the sensitivity of the model to the specificities of the training set. As λ increases, the model becomes less sensitive, and variance decreases.

The total error is the sum of $(\text{bias})^2$ and variance. The optimal value of λ is located at the minimum of this sum, representing the point where the model is complex enough to capture the signal but simple enough to ignore the noise.

Probabilistic model for regression

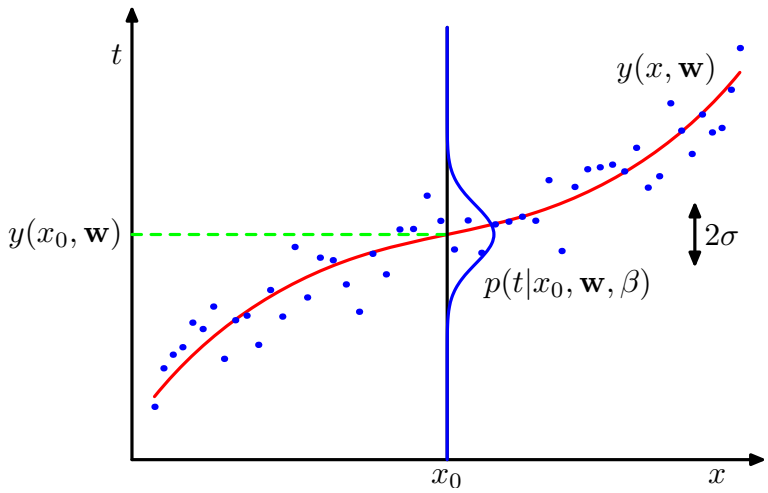
To move beyond simple function fitting and account for the inherent uncertainties in data, we now shift our focus to a probabilistic framework. This transition allows us to treat the relationship between inputs and targets not as a rigid deterministic rule, but as a class of joint probability distributions. By doing so, we can select the most probable model given our training data.

We begin by defining the class of joint distributions $p(\mathbf{x}, t) = p_M(\mathbf{x})p_C(t|\mathbf{x})$. In this context, we assume the distribution of inputs $p_M(\mathbf{x})$ is uniform, which allows us to concentrate entirely on the conditional distribution $p_C(t|\mathbf{x})$.

The core assumption of our model is that the observed target t, for a given input $\mathbf{x}$, is normally distributed around the value predicted by our hypothesis $h(\mathbf{x}; \overline{\mathbf{w}})$. This variability is captured by a Gaussian distribution with a variance σ^2 or, as it is often expressed in Bayesian literature, a **precision** β (where $\beta = 1/\sigma^2$). Mathematically, this is expressed as:

$$p(t|\mathbf{x}; \overline{\mathbf{w}}, \beta) = \mathcal{N}(t|h(\mathbf{x}; \overline{\mathbf{w}}), \beta^{-1})$$

This definition implies that our model $h(\mathbf{x}; \overline{\mathbf{w}})$ represents the mean of the distribution, while β represents our certainty: a higher precision means the observed data points are expected to cluster tightly around the model's prediction.



Maximum Likelihood Estimation (MLE)

Having established a probabilistic model, we need a criterion to estimate the parameters $\overline{\mathbf{w}}$ and the noise precision β. In a frequentist approach, we use the **Likelihood function**, which measures how well a specific set of parameters explains the observed training set $(\mathbf{X}, \mathbf{t})$. Assuming the observations are independent and identically distributed (i.i.d.), the likelihood is the product of the individual probabilities:

$$L(\overline{\mathbf{w}}, \beta|\mathbf{X}, \mathbf{t}) = p(\mathbf{t}|\mathbf{X}; \overline{\mathbf{w}}, \beta) = \prod_{i=1}^{n} \mathcal{N}(t_i|h(\mathbf{x}_i; \overline{\mathbf{w}}), \beta^{-1}) = \prod_{i=1}^{n} \frac{\sqrt{\beta}}{\sqrt{2\pi}} e^{-\frac{\beta}{2} r_i(\overline{\mathbf{w}})^2}$$

To simplify the optimization, we maximize the **Log-Likelihood** $l(\overline{\mathbf{w}}, \beta|\mathbf{X}, \mathbf{t})$, as the logarithm is a monotonic function that turns the product into a more manageable sum:

$$l(\overline{\mathbf{w}}, \beta|\mathbf{X}, \mathbf{t}) = \sum_{i=1}^{n} \log \mathcal{N}(t_i|h(\mathbf{x}_i; \overline{\mathbf{w}}), \beta^{-1})$$

By expanding the Gaussian definition and taking the log, we obtain:

$$l(\overline{\mathbf{w}}, \beta|\mathbf{X}, \mathbf{t}) = -\frac{\beta}{2} \sum_{i=1}^{n} r_i(\overline{\mathbf{w}})^2 + \frac{n}{2} \log \beta + c$$

where c is a constant independent of $\overline{\mathbf{w}}$ and β.

The maximization of this objective reveals a deep connection between probability and geometry:

- For the weights $\overline{\mathbf{w}}$, maximizing the log-likelihood is equivalent to maximizing $-\frac{1}{2} \sum r_i(\overline{\mathbf{w}})^2$, which is exactly the same as **minimizing the sum of squared errors**. This proves that Least Squares is the optimal frequentist estimator under the assumption of Gaussian noise.
- For the precision β, by setting the derivative with respect to β to zero:

$$\frac{\partial l}{\partial \beta} = -\frac{1}{2} \sum_{i=1}^{n} r_i(\overline{\mathbf{w}})^2 + \frac{n}{2\beta} = 0$$

We find that the maximum likelihood estimate for the noise variance is the average of the squared residuals:

$$\beta_{ML}^{-1} = \frac{1}{n}\sum_{i=1}^{n} r_i(\overline{\mathbf{w}})^2$$

As a result of this estimation, we obtain a **predictive distribution**. Instead of returning a single point estimate for a new input $\mathbf{x}$, the model now provides a full Gaussian density:

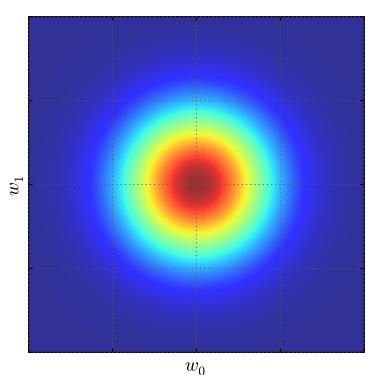
$$p(t|\mathbf{x}; \overline{\mathbf{w}}_{ML}, \beta_{ML}) = \mathcal{N}(t|h(\mathbf{x}; \overline{\mathbf{w}}_{ML}), \beta_{ML}^{-1})$$

The Bayesian Approach and Conjugate Priors

While MLE is powerful, it is strictly frequentist: it treats parameters as fixed, unknown constants and is highly susceptible to overfitting when the dataset is small. To introduce regularization within a probabilistic context, we move to the **Bayesian framework**. Here, parameters $\overline{\mathbf{w}}$ are treated as random variables with their own probability distributions.

Before seeing the data, we assume a **prior distribution** $p(\overline{\mathbf{w}}|\alpha)$. We typically choose a zero-mean Gaussian prior with a diagonal covariance matrix, governed by the hyperparameter α:

$$p(\overline{\mathbf{w}}|\alpha) = \mathcal{N}(\overline{\mathbf{w}}; \mathbf{0}, \alpha^{-1}\mathbf{I}) = \left(\frac{\alpha}{2\pi}\right)^{\frac{m}{2}} e^{-\frac{\alpha}{2}\overline{\mathbf{w}}^T\overline{\mathbf{w}}}$$



The choice of a Gaussian prior is strategically significant because the Gaussian is a **conjugate prior** for the mean of a Gaussian likelihood. This means that the resulting **posterior distribution**—the distribution of parameters after observing the training data—will also be Gaussian. According to Bayes' rule:

$$p(\overline{\mathbf{w}}|\mathbf{X}, \mathbf{t}; \alpha, \beta) \propto p(\mathbf{t}|\mathbf{X}, \overline{\mathbf{w}}; \beta)p(\overline{\mathbf{w}}|\alpha)$$

Given a general Gaussian prior $p(\overline{\mathbf{w}}) = \mathcal{N}(\overline{\mathbf{w}}; \mathbf{m}_0, \Sigma_0)$, the posterior is $p(\overline{\mathbf{w}}|\mathbf{X}, \mathbf{t}) = \mathcal{N}(\overline{\mathbf{w}}; \mathbf{m}_p, \Sigma_p)$ where:

$$\Sigma_p = (\Sigma_0^{-1} + \beta \overline{\mathbf{X}}^T \overline{\mathbf{X}})^{-1}$$
$$\mathbf{m}_p = \Sigma_p(\Sigma_0^{-1}\mathbf{m}_0 + \beta \overline{\mathbf{X}}^T \mathbf{t})$$

In our specific case, with a zero-mean prior ($\mathbf{m}_0 = \mathbf{0}$) and precision α ($\Sigma_0^{-1} = \alpha \mathbf{I}$), the posterior parameters simplify to:

$$\Sigma_p = (\alpha \mathbf{I} + \beta \overline{\mathbf{X}}^T \overline{\mathbf{X}})^{-1}$$
$$\mathbf{m}_p = \beta \Sigma_p \overline{\mathbf{X}}^T \mathbf{t}$$

Observe that the mean of the posterior $\mathbf{m}_p$ is exactly the same as the solution for Ridge Regression. In the Bayesian view, regularization is not just an arbitrary penalty added to the loss function; it is the natural consequence of assuming that the parameters themselves follow a Gaussian distribution centered at zero. As $\alpha \to 0$ (meaning we have no prior information), the posterior mean $\mathbf{m}_p$ converges back to the Maximum Likelihood estimate $\overline{\mathbf{w}}_{ML}$.

Sequential Bayesian Learning

A significant advantage of the Bayesian approach to linear regression is its inherent ability to process data incrementally. Unlike batch methods that require all data to be present simultaneously, Bayesian inference allows us to update our knowledge as new information arrives.

The core principle of sequential learning is that the posterior distribution obtained after observing a specific subset of data can serve as the prior distribution for the next phase of learning. This recursive update is a direct consequence of Bayes' Theorem.

When we receive a sequence of independent training sets $\mathcal{T}_1, \dots, \mathcal{T}_n$, we can update our beliefs about the parameter vector $\overline{\mathbf{w}}$ step-by-step. For the first dataset $\mathcal{T}_1$, the posterior is proportional to the product of the likelihood and the initial prior:

$$p(\overline{\mathbf{w}}|\mathcal{T}_1) \propto p(\mathcal{T}_1|\overline{\mathbf{w}})p(\overline{\mathbf{w}})$$

For any subsequent step k, the current posterior is updated using the new likelihood $p(\mathcal{T}_k|\overline{\mathbf{w}})$ and the previous posterior, which now acts as the prior:

$$p(\overline{\mathbf{w}}|\mathcal{T}_1, \dots, \mathcal{T}_k) \propto p(\mathcal{T}_k|\overline{\mathbf{w}})p(\overline{\mathbf{w}}|\mathcal{T}_1, \dots, \mathcal{T}_{k-1})$$

This chain of updates continues as more data is acquired:

$$p(\overline{\mathbf{w}}|\mathcal{T}_1, \dots, \mathcal{T}_n) \propto p(\mathcal{T}_n|\overline{\mathbf{w}})p(\overline{\mathbf{w}}|\mathcal{T}_1, \dots, \mathcal{T}_{n-1})$$

This "online" learning capability is extremely valuable in scenarios where data is collected in a stream or where the dataset is too large to fit into memory all at once. Every new observation refines our distribution, typically narrowing the variance and shifting the mean toward the true underlying parameters.

To visualize this process, let us consider a practical example of a simple linear regression model with one input variable x and one target variable t. The model is defined by two parameters: the slope w_1 and the intercept b (which we will also refer to as w_0):

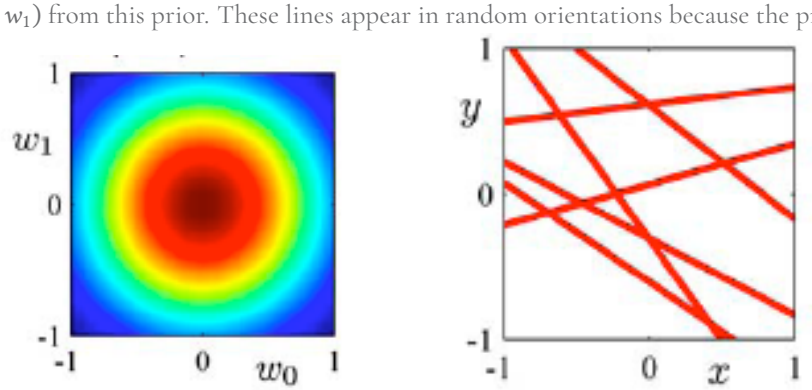
$$h(x; b, w_1) = w_1 x + b$$

The data for this example is generated from a "ground truth" linear function $y = a_0 + a_1 x$, where $a_0 = -0.3$ and $a_1 = 0.5$. The input values x are sampled uniformly from the interval $[-1, 1]$, and the targets are corrupted by adding zero-mean Gaussian noise with a standard deviation $\sigma = 0.2$.

Before observing any data, we must define our initial state of knowledge. We assume a bivariate Gaussian prior distribution $p(w_1, b)$ centered at the origin, reflecting our initial lack of a preferred direction or intercept:

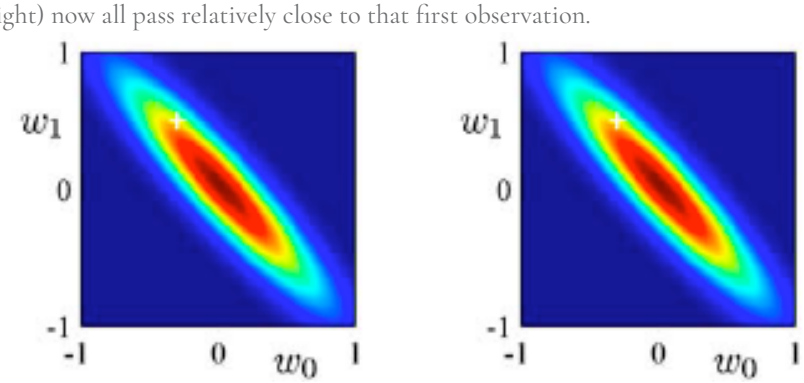
$$\boldsymbol{\mu} = \mathbf{0}, \quad \boldsymbol{\Sigma} = \sigma^2 \mathbf{I} = 0.04\mathbf{I}$$

Initial State: On the left, the prior distribution is a broad circle centered at $(0, 0)$. On the right, 6 lines are sampled by drawing weights (w_0, w_1) from this prior. These lines appear in random orientations because the prior is uninformative.



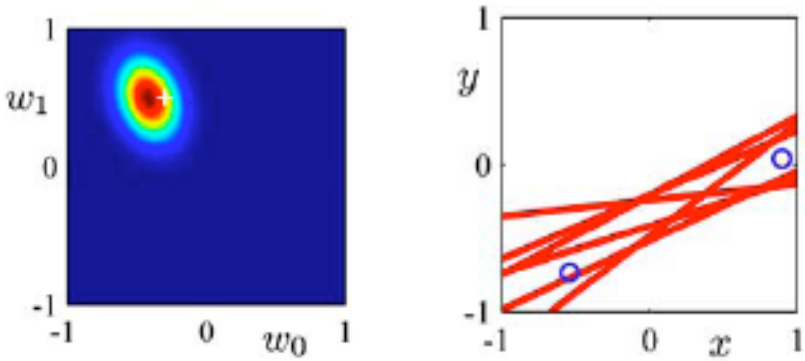
Once we observe the first item (x_1, t_1), represented by a circle in the right-hand plot, our knowledge is updated. The likelihood of this single point "carves" a path through the parameter space.

First Update: The posterior distribution $p(w_0, w_1 | x_1, t_1)$ (left) is no longer circular; it has been elongated into a stripe. This stripe represents all combinations of b and w_1 that could reasonably explain the single observed point. Consequently, the sampled lines (right) now all pass relatively close to that first observation.



As we observe a second item (x_2, t_2), the posterior is updated again. The intersection of the "knowledge" from the first point and the "knowledge" from the second point further constrains the parameters.

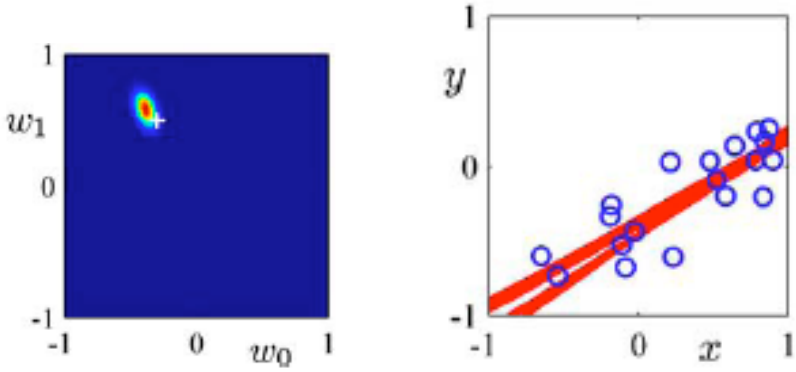
Second Update: The posterior (left) is now concentrated in the region where the requirements for both observed points are met. The sampled lines (right) are now much more restricted, as they must pass through the vicinity of both circles.



Finally, after observing a larger set of n items, the cumulative effect of the likelihood terms dominates the initial prior.

Final State:

- **Parameter Concentration:** As the number of observed items increases, the posterior distribution of w_0 and w_1 becomes highly concentrated. The variance decreases toward zero, and the distribution converges around the true parameter values $a_0 = -0.3$ and $a_1 = 0.5$.
- **Model Accuracy:** As a direct consequence, the lines sampled from this tight posterior are nearly indistinguishable from each other and are concentrated precisely around the true generating function $y = -0.3 + 0.5x$.



This visual evolution demonstrates how the Bayesian framework naturally handles the transition from high uncertainty (the broad prior) to high certainty (the sharp posterior) as evidence accumulates.

To understand why our certainty increases with more data, we can examine the evolution of the posterior covariance matrix Σ_n as we transition from n to $n+1$ observations.

Recall the general form of the posterior covariance matrix for a Gaussian prior and likelihood:

$$\Sigma_p^{-1} = \Sigma_0^{-1} + \beta \overline{\mathbf{X}}^T \overline{\mathbf{X}}$$

where Σ_0^{-1} is the prior precision, β is the noise precision of the data, and $\mathbf{\Phi}_n$ is the design matrix for n observations.

When a new observation $(\mathbf{x}_{n+1}, t_{n+1})$ is acquired, the new design matrix $\overline{\mathbf{X}}_{n+1}$ is simply the previous matrix $\overline{\mathbf{X}}_n$ with an additional row $\overline{\mathbf{x}}_{n+1}^T$ appended. The product $\overline{\mathbf{X}}_{n+1}^T\overline{\mathbf{X}}_{n+1}$ can be decomposed as:

$$\overline{\mathbf{X}}_{n+1}^T\overline{\mathbf{X}}_{n+1} = \overline{\mathbf{X}}_n^T\overline{\mathbf{X}}_n + \overline{\mathbf{x}}_{n+1}\overline{\mathbf{x}}_{n+1}^T$$

Substituting this into our update rule for the posterior precision Σ_{n+1}^{-1}, we get:

$$\Sigma_{n+1}^{-1} = \Sigma_n^{-1} + \beta\overline{\mathbf{x}}_{n+1}\overline{\mathbf{x}}_{n+1}^T$$

The term $\overline{\mathbf{x}}_{n+1}\overline{\mathbf{x}}_{n+1}^T$ is a positive semi-definite matrix. This means that with every new observation, we are adding a non-negative amount of information to the precision matrix. Since the precision increases, the "size" of the covariance matrix Σ (measured by its eigenvalues) must decrease.

To see how the actual variance decreases in a specific direction, we can use the **Sherman-Morrison formula** (a special case of the Woodbury matrix identity) to find the update for the covariance matrix itself:

$$\Sigma_{n+1} = \left(\Sigma_n^{-1} + \beta\overline{\mathbf{X}}\overline{\mathbf{X}}^T\right)^{-1} = \Sigma_n - \frac{\Sigma_n\overline{\mathbf{X}}\overline{\mathbf{X}}^T\Sigma_n}{\beta^{-1} + \overline{\mathbf{X}}^T\Sigma_n\overline{\mathbf{X}}}$$

From this expression, we can derive the change in variance:

$$\Delta\Sigma = \Sigma_n - \Sigma_{n+1} = \frac{\Sigma_n\overline{\mathbf{X}}\overline{\mathbf{X}}^T\Sigma_n}{\beta^{-1} + \overline{\mathbf{X}}^T x x\Sigma_n\overline{\mathbf{X}}}$$

Because the denominator is a scalar sum of positive terms (the noise variance and the projected parameter variance) and the numerator is a quadratic form $\mathbf{v}\mathbf{v}^T$ (where $\mathbf{v} = \Sigma_n\boldsymbol{\phi}$), the resulting matrix $\Delta\Sigma$ is **positive semi-definite**.

This proves that $\Sigma_{n+1} \preceq \Sigma_n$ in the sense of the Loewner partial ordering. Geometrically, the "uncertainty ellipsoid" in the parameter space can only shrink or remain the same size as more data is observed; it can never expand. This is exactly what we observed in the sequential learning example, where the broad circular prior collapsed into a tiny, dense point. This reduction in parameter variance is what ultimately leads to the narrowing of the uncertainty bands in the predictive distribution.

Maximum a Posteriori

Once we have established the posterior distribution $p(\overline{\mathbf{w}}|\mathbf{X}, \mathbf{t}; \alpha, \beta)$, we face the task of selecting a representative set of parameters from this distribution to make predictions. In the Bayesian framework, the most natural single-point representative is the **Maximum A Posteriori (MAP)** estimate, $\overline{\mathbf{w}}_{MAP}$. This vector represents the **mode** of the posterior distribution—the point where the probability density is at its highest.

To find the MAP estimate, we seek the value of $\overline{\mathbf{w}}$ that maximizes the posterior probability. Mathematically, it is more convenient to work with the logarithm of the probability, as it simplifies the product of the likelihood and the prior into a sum. Using Bayes' rule, the log-posterior is expressed as:

$$\log p(\overline{\mathbf{w}}|\mathbf{X}, \mathbf{t}; \alpha, \beta) = \log p(\mathbf{t}|\mathbf{X}, \overline{\mathbf{w}}, \beta) + \log p(\overline{\mathbf{w}}; \alpha) - \log p(\mathbf{t}|\mathbf{X}; \beta)$$

Since the term $p(\mathbf{t}|\mathbf{X};\beta)$ (the evidence) does not depend on $\overline{\mathbf{w}}$, it acts as a constant during the optimization process. Therefore, the maximization simplifies to:

$$\overline{\mathbf{w}}_{MAP} = \underset{\overline{\mathbf{w}}}{\operatorname{argmax}} \ (\log p(\mathbf{t}|\mathbf{X},\overline{\mathbf{w}};\beta) + \log p(\overline{\mathbf{w}};\alpha))$$

By simply changing the sign, we can reformulate this as a minimization problem, which is common in optimization literature:

$$\overline{\mathbf{w}}_{MAP} = \underset{\overline{\mathbf{w}}}{\operatorname{argmin}} \ (-\log p(\mathbf{t}|\mathbf{X},\overline{\mathbf{w}};\beta) - \log p(\overline{\mathbf{w}};\alpha))$$

To understand what this means in the case of gaussian model for regression considered here, we expand the terms using our Gaussian assumptions for both the likelihood and the prior. For the likelihood, we have:

$$\log p(\mathbf{t}|\mathbf{X},\overline{\mathbf{w}};\beta) = \sum_{i=1}^{n}\left(\frac{1}{2}\log\frac{\beta}{2\pi} - \frac{\beta}{2}r_i(\overline{\mathbf{w}})^2\right) = \frac{n}{2}\log\frac{\beta}{2\pi} - \frac{\beta}{2}\sum_{i=1}^{n} r_i(\overline{\mathbf{w}})^2$$

Similarly, for the Gaussian prior on the $d+1$ parameters (including the bias $\overline{w}_0$):

$$\log p(\overline{\mathbf{w}};\alpha) = \sum_{j=0}^{d}\left(\frac{1}{2}\log\frac{\alpha}{2\pi} - \frac{\alpha}{2}\overline{w}_j^2\right) = \frac{d+1}{2}\log\frac{\alpha}{2\pi} - \frac{\alpha}{2}\sum_{j=0}^{d}\overline{w}_j^2$$

By combining these and dropping the constants that do not involve $\overline{\mathbf{w}}$, we find that maximizing the posterior probability is equivalent to minimizing the following expression:

$$\frac{\beta}{2}\sum_{i=1}^{n} r_i(\overline{\mathbf{w}})^2 + \frac{\alpha}{2}\sum_{j=0}^{d}\overline{w}_j^2 \propto \frac{1}{2}\sum_{i=1}^{n} r_i(\mathbf{w})^2 + \frac{\alpha}{2\beta}\sum_{j=0}^{d}\overline{w}_j^2$$

This result is significant because it provides a probabilistic justification for **Ridge Regression**. The ratio of the two precision hyperparameters, $\lambda = \frac{\alpha}{\beta}$, corresponds exactly to the regularization hyperparameter λ we introduced earlier. We can now see that regularization is not just a heuristic to prevent overfitting; it is the direct consequence of assuming a Gaussian prior on the model parameters.

With the MAP estimate in hand, we can define the **predictive distribution** for a target value t given a new input $\mathbf{x}$. Just as in the Maximum Likelihood (ML) framework, the result is a Gaussian distribution centered at the model's prediction:

$$p(t|\mathbf{x},\overline{\mathbf{w}}_{MAP};\beta_{MAP}) = \mathcal{N}(t|h(\mathbf{x};\overline{\mathbf{w}}_{MAP}),\beta_{MAP}^{-1}) = \sqrt{\frac{\beta_{MAP}}{2\pi}}e^{-\frac{\beta_{MAP}}{2}(h(\mathbf{x},\overline{\mathbf{w}}_{MAP})-t)^2}$$

In this context, it can be shown that the MAP estimate for the precision, β_{MAP}, is equivalent to the β_{ML} value derived previously, representing the noise variance of the training data.

While the MAP estimate provides a more robust prediction than ML by incorporating prior knowledge, it still only uses a single point estimate of $\overline{\mathbf{w}}$. A fully Bayesian approach would instead integrate over all possible values of $\overline{\mathbf{w}}$ to account for parameter uncertainty.

Approaches to prediction in linear regression

Classical

A value $\overline{\mathbf{w}}_{LS}$ for $\overline{\mathbf{w}}$ is learned through a point estimate, performed by minimizing a quadratic cost function, or equivalently by maximizing likelihood (ML) under the hypothesis of gaussian noise; regularization can be applied to modify the cost function to limit overfitting.

Given any $\mathbf{x}$, the obtained value $\overline{\mathbf{w}}_{LS}$ is used to predict the corresponding t as $t = \overline{\mathbf{x}}^T \overline{\mathbf{w}}_{LS} = \mathbf{x}^T \mathbf{w}_{LS} + b_{LS}$.

Bayesian point estimation

The posterior distribution $p(\overline{\mathbf{w}}|\mathbf{X}, \mathbf{t}; \alpha, \beta)$ is derived and a point estimate is performed from it, computing the mode $\overline{\mathbf{w}}_{MAP}$ of the distribution (MAP).

This is equivalent to the classical approach, as $\overline{\mathbf{w}}_{MAP}$ corresponds to $\overline{\mathbf{w}}_{LS}$ if $\lambda = \frac{\alpha}{\beta}$. The prediction, for an element $\mathbf{x}$, is a gaussian distribution $\mathcal{N}(t|\overline{\mathbf{x}}^T \overline{\mathbf{w}}_{MAP}, \beta)$ for t, with mean $\overline{\mathbf{x}}^T \overline{\mathbf{w}}_{MAP} = \mathbf{x}^T \mathbf{w}_{MAP} + b_{MAP}$ and variance β^{-1}.

The distribution is not derived directly from the posterior $p(\overline{\mathbf{w}}|\mathbf{X}, \mathbf{t}; \alpha, \beta)$: it is built, instead, as a gaussian with mean depending from the expectation of the posterior, and variance given by the assumed noise.

Fully bayesian

The real interest is not in estimating $\overline{\mathbf{w}}$ or its distribution $p(\overline{\mathbf{w}}|\mathbf{X}, \mathbf{t}; \alpha, \beta)$, but in deriving the **predictive distribution** $p(t|\mathbf{x}, \mathbf{X}, \mathbf{t})$, which expresses our uncertainty about the target t for a new input $\mathbf{x}$, given the entire training history. This can be done computing the expectation of the probability $p(t|\mathbf{x}, \overline{\mathbf{w}}; \beta)$ predicted by a model instance wrt model instance distribution $p(\overline{\mathbf{w}}|\mathbf{X}, \mathbf{t}; \alpha, \beta)$, that is also by marginalization wrt to $\mathbf{w}$

$$p(t|\mathbf{x}, \mathbf{t}, \mathbf{X}; \alpha, \beta) = \int p(t|\mathbf{x}, \overline{\mathbf{w}}; \beta) p(\overline{\mathbf{w}}|\mathbf{t}, \mathbf{X}; \alpha, \beta) d\overline{\mathbf{w}}$$

This integral represents a convolution of two Gaussian distributions:

- $p(t|\mathbf{x}, \overline{\mathbf{w}}; \beta)$, the **likelihood** of a target given a specific $\overline{\mathbf{w}}$.
- $p(\overline{\mathbf{w}}|\mathbf{X}, \mathbf{t}; \alpha, \beta)$, the **posterior distribution** of the weights we derived previously.

The distribution $p(t|\mathbf{x}, \overline{\mathbf{w}}; \beta)$ is assumed gaussian, and $p(\overline{\mathbf{w}}|\mathbf{X}, \mathbf{t}, ; \alpha, \beta)$ is gaussian by the assumption that the likelihood $p(\mathbf{t}|\mathbf{X}, \overline{\mathbf{w}}; \beta)$ and the prior $p(\overline{\mathbf{w}}; \alpha)$ are gaussian themselves and by their being conjugate

$$p(t|\mathbf{x}, \mathbf{w}; \beta) = \mathcal{N}(t|\overline{\mathbf{w}}^T \overline{\mathbf{x}}, \beta)$$
$$p(\overline{\mathbf{w}}|\mathbf{X}, \mathbf{t}; \alpha, \beta) = \mathcal{N}(\overline{\mathbf{w}}; \beta \Sigma \overline{\mathbf{X}}^T \mathbf{t}, \Sigma)$$

where $\Sigma = (\alpha \mathbf{I} + \beta \overline{\mathbf{X}}^T \overline{\mathbf{X}})^{-1}$ is a $(d+1) \times (d+1)$ matrix.

Under such hypothesis, the predictive distribution is gaussian

$$p(t|\mathbf{x}, \mathbf{X}, \mathbf{t}; \alpha, \beta) = \mathcal{N}(t; m(\mathbf{x}), \sigma^2(\mathbf{x}))$$

with mean

$$m(\mathbf{x}) = \beta \overline{\mathbf{x}}^T \Sigma \overline{\mathbf{X}}^T \mathbf{t}$$

and variance

$$\sigma^2(\mathbf{x}) = \frac{1}{\beta} + \overline{\mathbf{x}}^T \Sigma \overline{\mathbf{x}}$$

The predictive variance $\sigma^2(\mathbf{x})$ is composed of two distinct parts that highlight the power of the Bayesian method:

1. $\frac{1}{\beta}$: This is the **intrinsic noise** in the data (aleatoric uncertainty). It represents the fundamental limit of predictability; even if we knew the true parameters perfectly, the noise ε would still exist.
2. $\overline{\mathbf{x}}^T \Sigma \overline{\mathbf{x}}$: This is the **parameter uncertainty** (epistemic uncertainty). It represents our lack of knowledge about the "true" $\overline{\mathbf{w}}$.

Example: evolution of the Bayesian Predictive Distribution

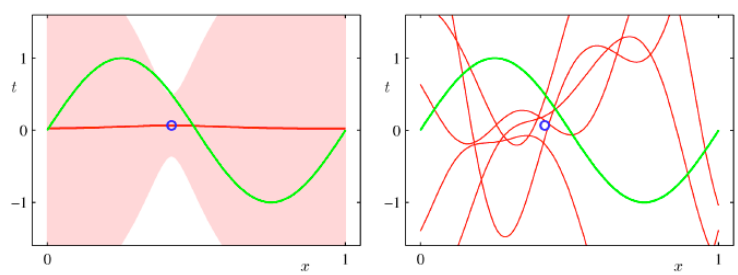
In this example, we examine how the Bayesian framework's understanding of the function $f(x) = \sin(2\pi x)$ evolves as it encounters increasing amounts of data. We utilize a model consisting of $m = 9$ Gaussian basis functions. This relatively high degree of complexity would typically lead to severe overfitting in a frequentist setting, but here we observe how the Bayesian predictive distribution manages uncertainty.

The following figures illustrate the model's state after observing training sets of $n = 1, 2, 4,$ and 25 items respectively. For each stage of the experiment, the visualization is divided into two parts:

- Left (Predictive Distribution): The blue circles represent the items in the training set, which are sampled uniformly from the input space with added Gaussian noise. The **red line** represents the expectation (the mean) of the predictive distribution $p(t|x, \mathbf{X}, \mathbf{t})$, effectively our current best guess for the function. The **pink shaded region** represents the variance, specifically showing one standard deviation from the mean. This captures the model's total uncertainty, including both the noise in the data and the uncertainty regarding the parameters.
- Right (Posterior Samples): Again, we see the training items, but here they are overlaid with **5 possible curves**. Each curve is generated by taking a random sample of weights $\overline{\mathbf{w}}$ from the current posterior distribution $p(\overline{\mathbf{w}}|\mathbf{X}, \mathbf{t}; \alpha, \beta)$. This provides a qualitative sense of the "candidate functions" that the model still considers plausible.

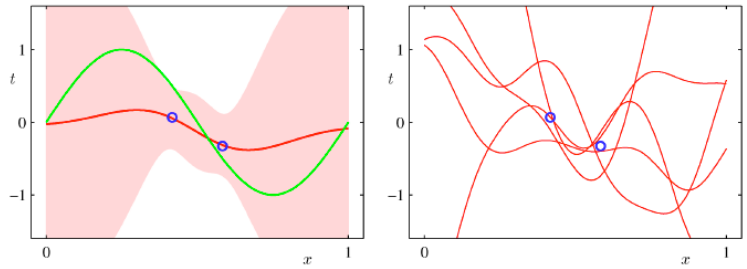
As we increase the number of observations n, we can follow the transition from ignorance to specialized knowledge.

The Case of $n = 1$. With only a single data point, the model has almost no information about the frequency or amplitude of the sine wave.



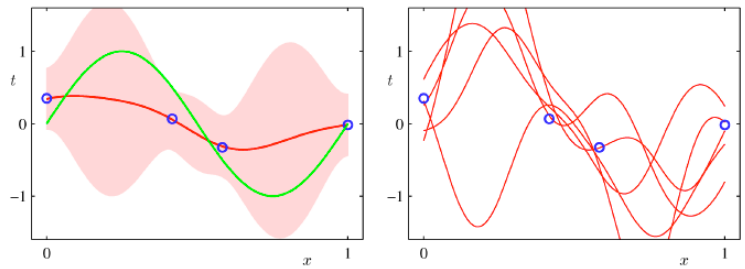
On the left, the predictive mean is essentially forced through the single point but remains nearly flat elsewhere. The pink shaded region is extremely wide, indicating high uncertainty across the entire domain. On the right, the 5 sampled curves are wildly different from each other, showing that many different functions (sloping up, sloping down, or staying flat) are equally probable given only one point.

The Case of $n = 2$. Adding just one more point begins to constrain the possible orientations of the function.



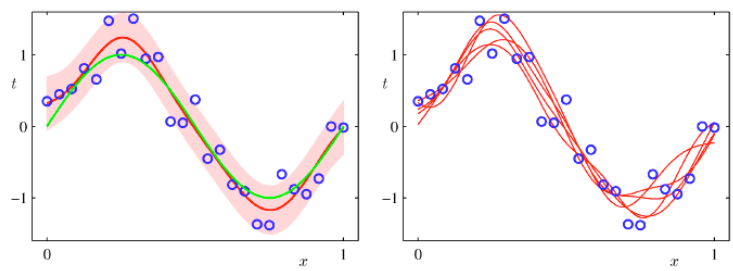
As we see, the expectation (red line) now attempts to connect the two points. However, the variance (pink shade) is still quite large, particularly in the regions far from these two observations. The sampled curves on the right start to align slightly but still show a vast range of possible shapes.

The Case of $n = 4$. With four points, the characteristic "S" shape of the sine wave begins to emerge.



An important observation here is that the pink uncertainty band is not uniform. It "pinches" or narrows at the locations of the training items and widens in the gaps between them. This shows the model's confidence is localized; it knows the function better where it has seen data. The sampled curves on the right are now becoming more consistent, all sharing the same general oscillation.

The Case of $n = 25$. With twenty-five points, the model has gathered sufficient evidence to largely resolve the underlying signal.



At $n = 25$, the predictive mean (red line) is an excellent approximation of $f(x) = \sin(2\pi x)$. Most importantly, the variance (pink shade) has collapsed significantly and is now nearly uniform across the domain. The 5 sampled curves on the right are now so similar that they are almost indistinguishable, indicating that the posterior distribution has become very peaky (low variance) around the optimal weights.

In this specific example, we see the Bayesian power to provide not just a curve, but a "margin of error" that naturally shrinks as the training set grows.

The predictive distribution $p(t|\mathbf{x}, \mathbf{t})$ provides a Gaussian measure of the target t for a new input $\mathbf{x}$. Its variance, $\sigma_n^2(\mathbf{x})$, defines the "pink shade" observed in our previous plots. As observed before, this variance is the sum of two distinct components:

$$\sigma_n^2(\mathbf{x}) = \underbrace{\frac{1}{\beta}}_{\text{Noise}} + \underbrace{\overline{\mathbf{x}}^T \Sigma \overline{\mathbf{x}}}_{\text{Model Uncertainty}}$$

The term $\overline{\mathbf{x}}^T \Sigma \overline{\mathbf{x}}$ is a quadratic form that measures how much the current parameter uncertainty "projects" onto the prediction at point $\mathbf{x}$.

As we saw in the sequential learning example, the precision matrix $\Sigma \overline{\mathbf{x}}^{-1}$ is updated by adding the outer products of the basis vectors of the training points:

$$\Sigma_n^{-1} = \Sigma_0^{-1} + \beta \sum_{i=1}^{n} \overline{\mathbf{x}}\overline{\mathbf{x}}^T$$

When we evaluate the model at a new point $\mathbf{x}$ that is very close to a training point $\mathbf{x}_i$, the vector $\overline{\mathbf{x}}$ becomes highly aligned with the direction in which we have already gained significant information (the eigenvectors of Σ^{-1} with large eigenvalues). Mathematically, this causes the quadratic form $\overline{\mathbf{x}}^T \Sigma \overline{\mathbf{x}}$ to decrease significantly.

Conversely, in "data-void" regions where $\mathbf{x}$ is far from any $\mathbf{x}_i$, the vector $\overline{\mathbf{x}}$ points in a direction where the precision matrix has not been "strengthened" by the summation. Consequently, the term remains large, resulting in the wider pink bands between the blue training points.

The model is effectively saying: "I have seen samples in this neighborhood, so my parameters are well-constrained for this specific type of input; however, for inputs I haven't seen, my parameter uncertainty remains high." This is why, in the $n = 25$ plot, the pink shade is almost uniform: the basis functions are sufficiently 'covered' by data points across the entire domain.

Fully bayesian regression and hyperparameter marginalization

In a fully Bayesian treatment, we should ideally marginalize over the hyperparameters α and β by defining hyperpriors.

$$p(t|\mathbf{x}, \mathbf{X}, \mathbf{t}) = \int p(t|\mathbf{x}, \overline{\mathbf{w}}; \beta) p(\overline{\mathbf{w}}|\mathbf{X}, \mathbf{t}; \alpha, \beta) p(\alpha, \beta|\mathbf{X}, \mathbf{t})\, d\overline{\mathbf{w}}\, d\alpha\, d\beta$$

where, as seen before,

- $p(t|\mathbf{x}, \overline{\mathbf{w}}, \beta) = \mathcal{N}(t; \overline{\mathbf{w}}^T \overline{\mathbf{x}}, \beta)$
- $p(\overline{\mathbf{w}}|\mathbf{X}, \mathbf{t}, \alpha, \beta) = \mathcal{N}(\overline{\mathbf{w}}; \mathbf{m}_N, \Sigma)$, with $\Sigma = (\alpha \mathbf{I} + \beta \overline{\mathbf{X}}^T \overline{\mathbf{X}})^{-1}$ e $\mathbf{m}_N = \beta \Sigma \overline{\mathbf{X}}^T \mathbf{t}$

this marginalization wrt $\mathbf{w}, \alpha, \beta$ is analytically intractable: we may however consider approximation methods.

Approximating the Hierarchical Bayesian Integral

To finalize our discussion on the Bayesian treatment of linear regression, we address the situation where the marginalization over hyperparameters becomes a necessity. In a fully hierarchical Bayesian model, we do not simply select fixed values for α and β. Instead, we ideally integrate them out to ensure that our uncertainty about the model's complexity and noise levels is reflected in our final predictions.

The goal is to compute the **predictive distribution** by accounting for the uncertainty in both the parameters $\overline{\mathbf{w}}$ and the hyperparameters α, β:

$$p(t|\mathbf{x}, \mathbf{X}, \mathbf{t}) = \int p(t|\mathbf{x}, \overline{\mathbf{w}}; \beta) p(\overline{\mathbf{w}}|\mathbf{X}, \mathbf{t}; \alpha, \beta) p(\alpha, \beta|\mathbf{X}, \mathbf{t})\, d\overline{\mathbf{w}}\, d\alpha\, d\beta$$

In this expression, we recognize the following components:

- The Likelihood $p(t|\mathbf{x}, \overline{\mathbf{w}}, \beta) = \mathcal{N}(t; \overline{\mathbf{w}}^T \overline{\mathbf{x}}), \beta^{-1})$, defining the distribution of the target around the model's prediction.
- The Weight Posterior $p(\overline{\mathbf{w}}|\mathbf{X}, \mathbf{t}, \alpha, \beta) = \mathcal{N}(\overline{\mathbf{w}}; \mathbf{m}_N, \Sigma)$, where $\Sigma = (\alpha \mathbf{I} + \beta \overline{\mathbf{X}}^T \overline{\mathbf{X}})^{-1}$ and $\mathbf{m}_N = \beta \Sigma \overline{\mathbf{X}}^T \mathbf{t}$.
- The Hyperparameter Posterior: $p(\alpha, \beta|\mathbf{X}, \mathbf{t})$, representing our updated belief about regularization and noise after observing the training data.

The non-linear dependence of the weight posterior (specifically the mean $\mathbf{m}_N$ and covariance Σ) on the hyperparameters prevents the evaluation of the integral in a closed form. To deal with this, we employ various approximation methods can be employed.

The Laplace Approximation. The Laplace approximation is a deterministic method used to approximate a complicated, non-Gaussian distribution with a Gaussian one. It works by finding the **mode** of the distribution (the MAP estimate) and performing a second-order Taylor expansion of the log-probability around that mode.

Mathematically, if we have a distribution $p(\mathbf{z}) = f(\mathbf{z})/Z$, we find the mode $\mathbf{z}_0$ and approximate the log-density as:

$$\ln f(\mathbf{z}) \approx \ln f(\mathbf{z}_0) - \frac{1}{2}(\mathbf{z} - \mathbf{z}_0)^T \mathbf{A} (\mathbf{z} - \mathbf{z}_0)$$

where $\mathbf{A}$ is the **Hessian matrix** (the matrix of second derivatives) of the negative log-probability. This effectively replaces the intractable hyperparameter posterior with a localized Gaussian, making the subsequent integration more manageable.

Variational Inference (VI). Variational methods cast the inference problem as an optimization task rather than an integration task. We define a family of simpler distributions $q(\overline{\mathbf{w}}, \alpha, \beta)$ - often assuming they are independent, a technique known as the *mean-field approximation* - and minimize the **Kullback-Leibler (KL) divergence** between this proxy and the true, intractable posterior.

Markov Chain Monte Carlo (MCMC). MCMC is a stochastic approach that bypasses the need for analytical integration entirely. By constructing a Markov chain that has the posterior $p(\overline{\mathbf{w}}, \alpha, \beta|\mathbf{X}, \mathbf{t})$ as its equilibrium distribution, we can draw a series of samples $\{\overline{\mathbf{w}}^{(l)}, \alpha^{(l)}, \beta^{(l)}\}_{l=1}^{L}$. The predictive distribution is then approximated by the empirical average:

$$p(t|\mathbf{x}, \mathbf{X}, \mathbf{t}) \approx \frac{1}{L} \sum_{l=1}^{L} p(t|\mathbf{x}, \overline{\mathbf{w}}^{(l)}, \beta^{(l)})$$

As the number of samples $L \to \infty$, this numerical approximation converges to the true value of the integral. While computationally intensive, it is the most flexible method for complex hierarchical models.

The choice between these methods depends on the size of the dataset and the required precision. In many linear regression contexts, the **Evidence Framework** described below (which provides a point-estimate) is sufficient, but for highly non-linear or safety-critical applications, MCMC or Variational methods are preferred to ensure the full breadth of uncertainty is captured.

Would you like me to demonstrate how **Gibbs Sampling**—a specific MCMC algorithm—would be implemented to sample from the posterior of $\overline{\mathbf{w}}$, α, and β in this model?

The Evidence Framework (Empirical Bayes)

Up to this point, we have treated the hyperparameters α and β as quantities that must be chosen externally, either by manual tuning or by computationally expensive procedures such as cross-validation. While these approaches can work in practice, they are conceptually unsatisfactory: they separate model training from hyperparameter selection and require additional data or repeated training cycles.

A more principled and elegant solution is provided by the **evidence framework**, also known as **Empirical Bayes**. The key idea is to treat α and β not as arbitrary tuning parameters, but as quantities that can be inferred directly from the data in a probabilistically consistent manner.

Rather than selecting α and β by hand, we choose the values that maximize the **marginal likelihood** $p(\mathbf{t}|\alpha, \beta)$, commonly called the **evidence**. This quantity measures how well the model, with given hyperparameters, explains the observed data after integrating out the model parameters $\overline{\mathbf{w}}$.

Formally, the evidence is defined as

$$p(\mathbf{t}|\alpha, \beta) = \int p(\mathbf{t}|\overline{\mathbf{w}}, \beta)\, p(\overline{\mathbf{w}}|\alpha)\, d\overline{\mathbf{w}}.$$

This integral expresses a crucial Bayesian idea: instead of conditioning on a single parameter vector $\overline{\mathbf{w}}$, we average over all possible parameter values, weighted by their prior probability. In this sense, the evidence evaluates how plausible the data are under the entire model class defined by α and β, rather than under one specific fitted model.

Because both the likelihood $p(\mathbf{t}|\overline{\mathbf{w}}, \beta)$ and the prior $p(\overline{\mathbf{w}}|\alpha)$ are Gaussian, this integral can be solved analytically. Taking the logarithm of the result leads to the **log-evidence**:

$$\log p(\mathbf{t}|\alpha, \beta) = \frac{m}{2} \log \alpha + \frac{n}{2} \log \beta - E(\mathbf{m}_p) - \frac{1}{2} \log |\mathbf{A}| - \frac{n}{2} \log(2\pi).$$

This expression contains several competing terms. The first two favor large values of α and β, while the remaining terms penalize overly complex or poorly fitting models. Maximizing the log-evidence therefore implements an automatic trade-off between data fit and model complexity.

Two central quantities appear in this formula:

- $\mathbf{A} = \Sigma^{-1} = \alpha\mathbf{I} + \beta\overline{\mathbf{X}}^T\overline{\mathbf{X}}$ is the **Hessian matrix** of the error function. It is the inverse of the posterior covariance matrix and quantifies the curvature of the log-posterior around its maximum. Intuitively, a larger $\mathbf{A}$ means a sharper, more concentrated posterior, indicating greater certainty about the parameter values.
- $E(\mathbf{m}_p) = \frac{\beta}{2}\|\mathbf{t} - \overline{\mathbf{X}}\mathbf{m}_p\|^2 + \frac{\alpha}{2}\mathbf{m}_p^T\mathbf{m}_p$ is the total regularized error evaluated at the posterior mean $\mathbf{m}_p$. The first term measures how well the model fits the data (residual error), while the second term penalizes large weights, enforcing smoothness and preventing overfitting.

From the Hessian to the Effective Number of Parameters

To determine the optimal hyperparameters, we maximize the log-evidence with respect to α and β. In doing so, a fundamental concept naturally emerges: the **effective number of parameters**, denoted by γ.

The total precision of the posterior is governed by the Hessian matrix

$$\mathbf{A} = \alpha\mathbf{I} + \beta\overline{\mathbf{X}}^T\overline{\mathbf{X}}.$$

The term $\alpha\mathbf{I}$ represents the contribution of the prior, while $\beta\overline{\mathbf{X}}^T\overline{\mathbf{X}}$ encodes the information coming from the data. To understand their relative influence, we examine the eigenvalues η_i of the data-dependent matrix $\beta\overline{\mathbf{X}}^T\overline{\mathbf{X}}$.

For each direction i in parameter space, we define the ratio

$$\lambda_i = \frac{\eta_i}{\alpha + \eta_i}.$$

This quantity behaves like a soft switch that tells us whether a parameter is mainly controlled by the data or by the prior.

- **Data Dominance** ($\eta_i \gg \alpha$): When the data provide strong information in direction i, the ratio λ_i approaches 1. In this case, the prior has little influence, and the parameter is effectively determined by the data.
- **Prior Dominance** ($\eta_i \ll \alpha$): When the data are weak or noisy in direction i, the ratio λ_i approaches 0. Here, the prior dominates and effectively shrinks the corresponding parameter toward zero.

The total effective number of parameters is then defined as the sum of these contributions across all m dimensions:

$$\gamma = \sum_{i=1}^{m} \frac{\eta_i}{\alpha + \eta_i}.$$

This quantity has a clear interpretation: it counts how many independent directions in parameter space are truly informed by the data.

- If $\gamma \approx m$, the dataset contains enough information to determine nearly all parameters, and regularization plays only a minor role.

- If $\gamma \ll m$, most parameters are constrained by the prior, meaning that the model is effectively using far fewer degrees of freedom than its nominal dimension.

A useful intuition is given by the spring analogy. Imagine each weight as attached to two springs: one pulling toward zero (the prior, stiffness α) and one pulling toward the data-driven estimate (stiffness η_i). If the data spring is much stronger, the weight moves toward the data; if the prior spring dominates, the weight remains near zero.

The quantity γ tells us, in aggregate, how many weights have been significantly pulled away from zero by the data. This provides the conceptual foundation for automatically adapting model complexity using the Evidence Framework.

Estimation of the Optimal α and β

Setting the derivatives of the log-evidence with respect to α and β to zero leads to two re-estimation formulas.

- The optimal α is inversely related to the average squared magnitude of the effective weights:

$$\alpha = \frac{\gamma}{\mathbf{m}_p^T \mathbf{m}_p}.$$

  This means that if the posterior weights are large, we reduce α (weaker regularization), whereas if they are small, we increase α to prevent overfitting.
- The optimal β is determined by the residual variance, corrected for the effective degrees of freedom used by the model:

$$\frac{1}{\beta} = \frac{1}{n-\gamma} \sum_{i=1}^{n} (t_i - \mathbf{m}_p^T \overline{\mathbf{x}}_i)^2.$$

  The term $n - \gamma$ plays the role of an effective sample size, accounting for the fact that γ parameters have been used to explain the data.

Since both $\mathbf{m}_p$ and γ depend on α and β, these equations must be solved iteratively. The procedure is:

1. Initialize α and β to reasonable starting values.
2. Compute the posterior mean $\mathbf{m}_p$ and the effective number of parameters γ.
3. Update α and β using the formulas above.
4. Repeat until convergence.

This iterative scheme typically converges quickly and provides a fully automated way to balance model complexity and data fit.

The Evidence Framework implements an automatic form of complexity control. If the data are noisy (small β), the evidence is maximized by increasing α, which shrinks the weights toward zero. If the data are clean and informative, γ approaches m, allowing the model to use its full expressive power.

The major advantage is that this procedure relies solely on the training data, eliminating the need for a separate validation set or expensive cross-validation. In this sense, Empirical Bayes provides a self-consistent and computationally efficient method for model selection within a Bayesian framework.